Case Study on the Analysis of SDLC Models Applied to a Real-World Software Application

AIM

To analyze the Software Development Life Cycle (SDLC) models used in the development of a real-world healthcare software application and identify the most suitable SDLC model for ensuring clinical compliance and operational optimization.

OBJECTIVE

- Study different software development lifecycles (SDLC) and deployment paradigms.
- Analyze the specific architectural development workflow of a real-world medical visualization tool.
- Compare structural tradeoffs, deployment speeds, and data compliance limitations across platforms.
- Recommend the single most appropriate development framework tailored explicitly for medical dashboards.

SELECTED SOFTWARE APPLICATION

Healthcare Operations: Hospital Patient Triage & Resource Allocation Dashboard

OVERVIEW

A Real-Time Medical Triage & Resource Allocation Dashboard allows clinical coordinators and administrative teams to:

- Register, authenticate, and securely log in medical personnel based on roles.
- Sort arriving emergency patients via a standardized automated visual triage system.
- Allocate and dispatch immediate critical care resources (ICU beds, staff shifts, specialized ventilators).
- Monitor dynamic wait-time fluctuations across individual operational wings.
- Evaluate localized department performance matrices via predictive analytics interfaces.

FUNCTIONAL REQUIREMENTS

Clinical Triage Module:

- Automated Patient Intake Intake Tracking and Emergency Sorting Logging.
- Severity Index Evaluation (ESI level tags matching dynamic algorithms).
- Dynamic Wait-Time Trackers updating operational counters globally.

Resource Management Module:

- Real-time ICU, Ward, and Operating Room Occupancy Matrix Logs.
- Medical Personnel Rota Shifts Management and Automated Dispatch System.
- Ventilator, Defibrillator, and Crash Cart Availability Mapping.

Administrative Workspace Module:

- Enterprise HIPAA Security & Role-Based Access Control Matrices.
- Cross-Departmental Metric Feeds and Aggregated Clinical Overlays.
- Dynamic PDF Audit Log Generation and Incident Analytical Data Reports.

SDLC MODELS ANALYSIS

SDLC Model	Suitability	Advantages	Limitations
Waterfall	Low	Clear phases; simple document tracking structure.	Highly rigid; cannot adapt to changing real-time metrics.
Prototype	Medium	Excellent for refining UI/UX wireframes with clinicians.	Increases code technical debt and architecture cost curves.
Spiral	High	Superior risk assessment for data safety validation.	Highly complex; slow release cadences delay deployment.
Agile	Very High	Continuous feedback iteration loops optimize visual modules.	Demands extensive medical domain expertise from engineers.
DevOps	Very High	Continuous automated validation ensures uninterrupted uptime.	Significant initial configuration of security tooling pipelines.

WHY AGILE IS BEST?

Key Reasons:

- Clinical workflows and sorting priorities change frequently based on hospital load.
- New predictive analytical models must be deployed iteratively without rewriting infrastructure.
- Urgent interface bug fixes can be integrated and deployed into active environments instantly.
- Incorporating active medical staff feedback dramatically reduces error frequencies in critical views.

AGILE DEVELOPMENT PROCESS

Visual Lifecycle: Requirement Gathering → Sprint Planning → Design → Development → Testing → Deployment → Feedback Loop

SDLC PHASES

1. Requirement Analysis: Documenting core UI filters, data connection protocols for electronic health records (EHR), database schemas, and clinical user stories.
Deliverable: Formal Software Requirement Specification (SRS) Document.

2. System Design: Visual UI layout design wireframing, role-based schema security definitions, high-throughput API gateway routing, and low-latency database queries.
Deliverable: Complete Database Schema Diagrams and Component Layouts.

3. Development: Code generation utilizing component building libraries to compile modules (Triage Grid, Room Allocator, Administrative Logs).
Frameworks: Low-Code Components / Python Analytics / Modern Web Runtimes.
Database: Enterprise Relational Layer (PostgreSQL / MySQL).

4. Testing: Multi-tier validations checking critical data pathways under heavy loads.
Testing Types: Unit Testing, End-to-End Integration, UI Load Stress Testing, User Acceptance Testing (UAT).

Test Case Target	Expected System Verification Result
Staff Secure Authentication	Successful validation; role-restricted dashboard layout rendered.
Invalid User Access Attempt	System access rejected; secure access denied error logged.
Automated ESI Level Sorting	Patient sorted accurately; real-time queue rankings adjusted.
ICU Bed Space Allocation	Live capacity matrix updated; visual asset highlighted green.

5. Deployment: Staging onto cloud clusters using fully containerized web pipelines.
Cloud Infrastructure: High Availability Nodes (AWS / Azure / Google Cloud Platforms).

6. Maintenance: Patching visual elements, refining pipeline security groups, and maximizing chart query performance parameters.
Activities: Uptime Optimization, Security Hardening, Feature Expansion.

COMPARISON OF SDLC MODELS

Criteria Metric	Waterfall	Prototype	Spiral	Agile	DevOps
Req. Changes	Poor	Good	Excellent	Excellent	Excellent
Staff Feedback	Low	High	High	Very High	Very High
Risk Mitigation	Low	Medium	High	Medium	High
Release Speed	Slow	Medium	Medium	Fast	Very Fast
Capital Cost	Low	Medium	High	Medium	Medium
App Suitability	No	Partially	Yes	Best	Best

RESULTS

The Medical Triage and Capacity Dashboard environment requires constant graphical updates, zero interface lag, and immediate workflow adjustments. Agile processes drive iterative layout designs while DevOps automation secures continuous code integration and security testing. This combination provides the most optimal framework for dynamic healthcare tracking systems.

CONCLUSION

Selecting an appropriate lifecycle strategy depends on the dynamic volatility of system requirements. For high-stakes, data-rich clinical interfaces like triage dashboards, Agile provides rapid flexibility and continuous staff involvement, while DevOps guarantees high operational efficiency and automated deployment safety. This structural alliance delivers maximum software reliability and optimized hospital resource throughput.

This is for informational purposes only. For medical advice or diagnosis, consult a professional. AI responses may include mistakes.